Shortest Path Problem

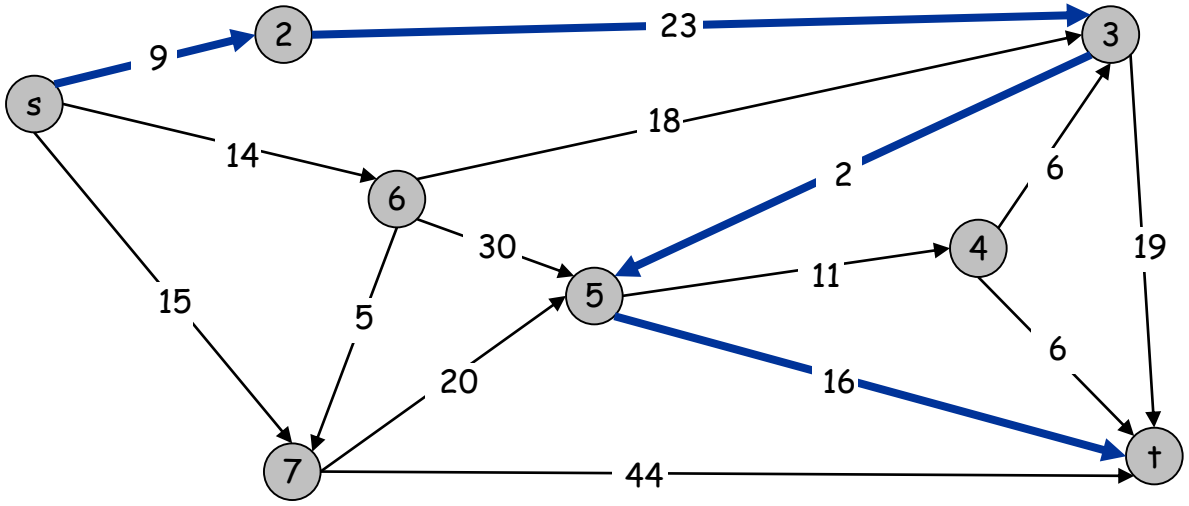
Shortest path network.

- Directed graph $G = (V, E)$.
- Source s, destination d.

Shortest path problem: find shortest directed path from s to d.

↑

cost of path = sum of edge costs in path



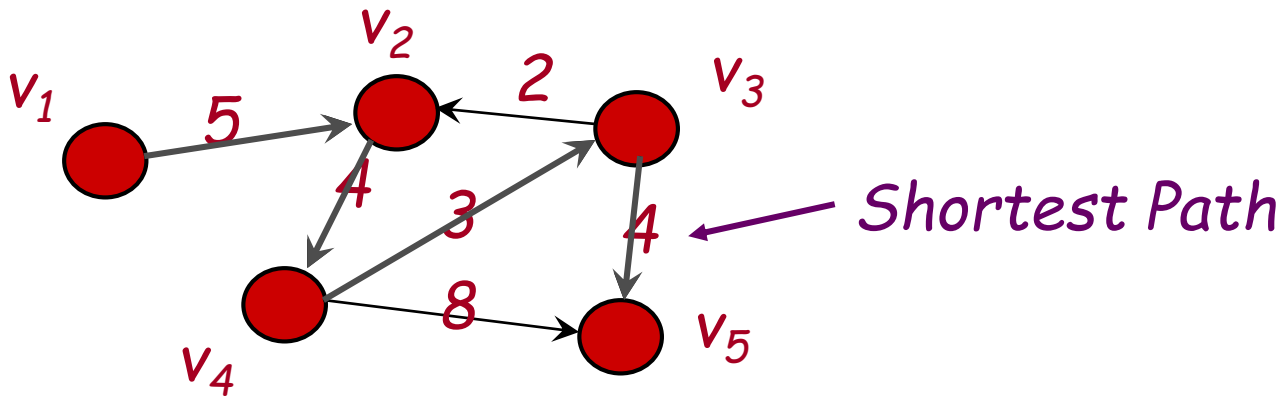
Cost of path s-2-3-5-d
= 9 + 23 + 2 + 16
= 48.

Formal definition of shortest path

Given a weighted directed graph G.
Let P be a path of G from x to y.

cost(P) = $\sum_{e \in P}$weight(e)

The shortest path is a path P which minimizes cost(P)



Dijkstra's Algorithm

Dijkstra's algorithm is an iterative procedure that finds the shortest path between vertices u and v in a weighted graph.

It proceeds by finding the length of the shortest path from u to successive vertices and adding these vertices to a distinguished set of vertices S.

The algorithm terminates once it reaches the terminal vertex.

Dijkstra's algorithm

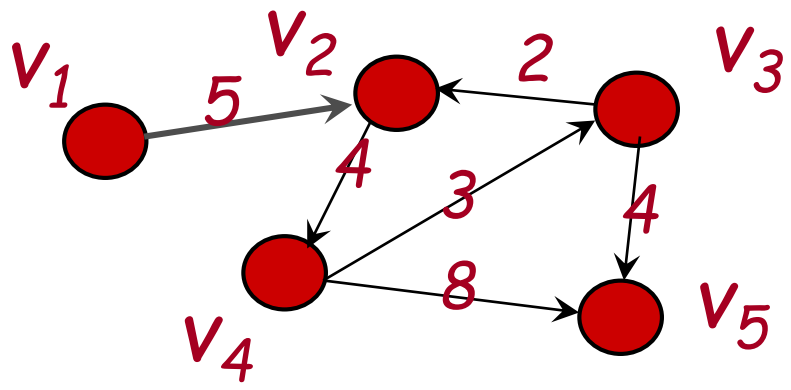
Consider a graph G, each edge (u, v) has a weight $w(u, v) > 0$.

Suppose we want to find the shortest path starting from v_1 to any node v_i

Let S be a subset of nodes in G.

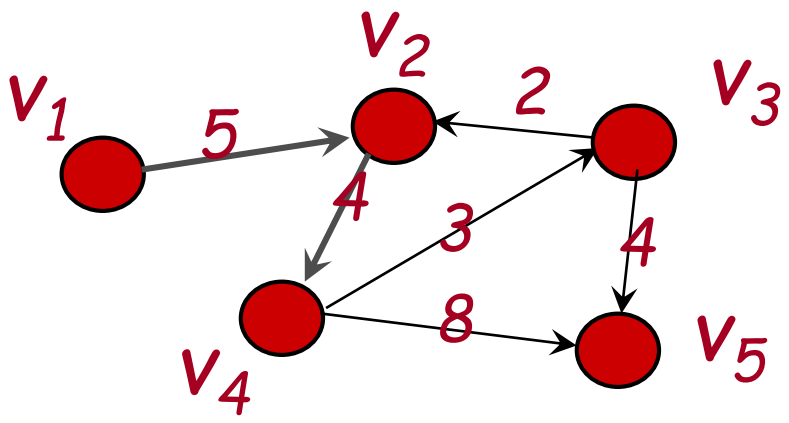
Let $cost[v_i]$ be the weight of the shortest path from v_1 to v_i that passes through nodes in S.

Dijkstra's algorithm - example



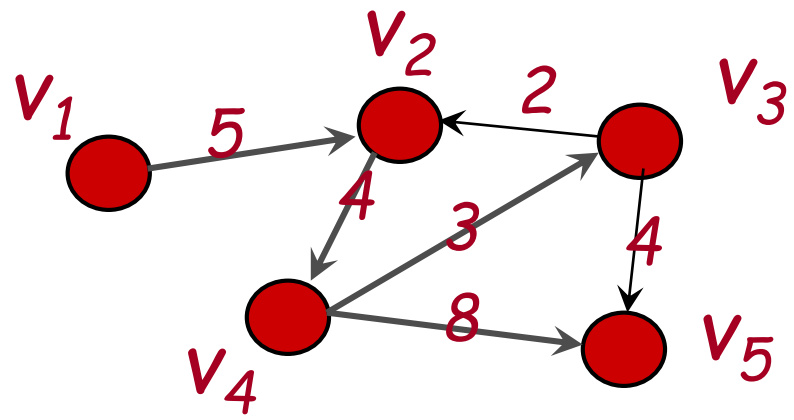
	v	S	cost[v_1]	cost[v_2]	cost[v_3]	cost[v_4]	cost[v_5]
1		{v_1}	0	5	∞	∞	∞

Dijkstra's algorithm - example



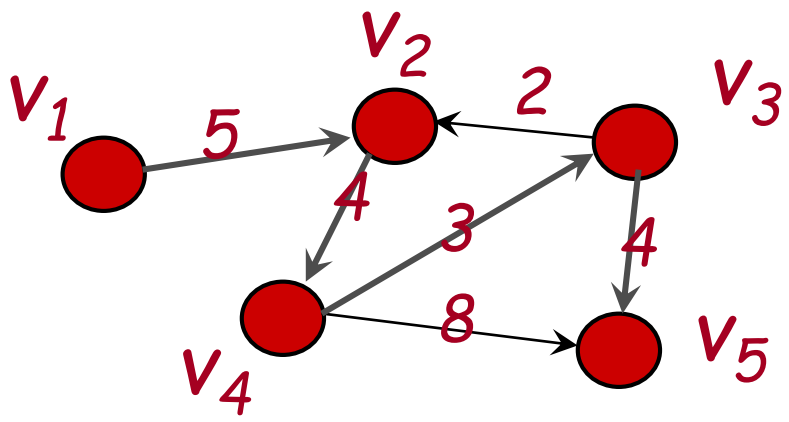
	v	S	cost[v_1]	cost[v_2]	cost[v_3]	cost[v_4]	cost[v_5]
1		$\{v_1\}$	0	5	∞	∞	∞
2	v_2	$\{v_1, v_2\}$	0	5	∞	9	∞

Dijkstra's algorithm - example



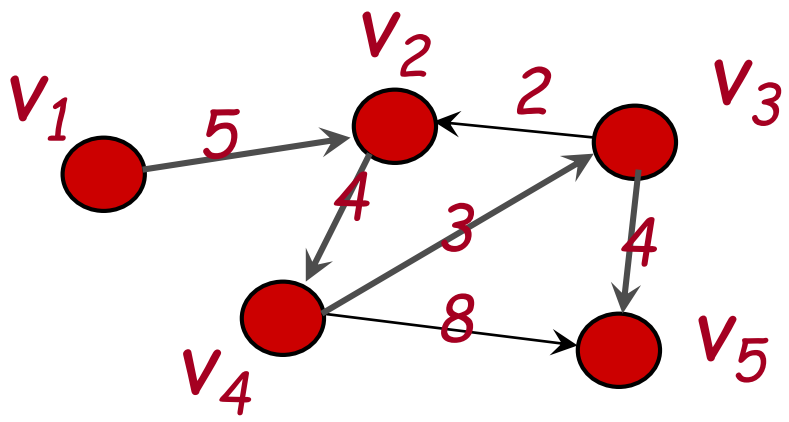
	v	S	cost[v_1]	cost[v_2]	cost[v_3]	cost[v_4]	cost[v_5]
1		{v_1}	0	5	∞	∞	∞
2	v_2	{v_1, v_2}	0	5	∞	9	∞
3	v_4	{v_1, v_2, v_4}	0	5	12	9	17

Dijkstra's algorithm - example



	v	S	cost[v_1]	cost[v_2]	cost[v_3]	cost[v_4]	cost[v_5]
1		$\{v_1\}$	0	5	∞	∞	∞
2	v_2	$\{v_1, v_2\}$	0	5	∞	9	∞
3	v_4	$\{v_1, v_2, v_4\}$	0	5	12	9	17
4	v_3	$\{v_1, v_2, v_4, v_3\}$	0	5	12	9	16

Dijkstra's algorithm - example



	v	S	cost[v_1]	cost[v_2]	cost[v_3]	cost[v_4]	cost[v_5]
1		$\{v_1\}$	0	5	∞	∞	∞
2	v_2	$\{v_1, v_2\}$	0	5	∞	9	∞
3	v_4	$\{v_1, v_2, v_4\}$	0	5	12	9	17
4	v_3	$\{v_1, v_2, v_4, v_3\}$	0	5	12	9	16
5	v_5	$\{v_1, v_2, v_4, v_3, v_5\}$	0	5	12	9	16

Dijkstra's algorithm

```
SP-Dijkstra( )
 n = number of nodes in the graph;
 for i = 1 to n
   cost[v_i] = w(v_1, v_i) ;
   S = { v_1 } ;
 for j = 2 to n {
   find the smallest cost[v_i] s.t. v_i is not in S;
   include v_i to S;
   for (all nodes v_j not in S) {
        if (cost[v_j] > cost[v_i] + w(v_i, v_j))
              cost[v_j] = cost[v_i] + w(v_i, v_j);
   }
 }
```

Dijkstra's Algorithm : Undirected graph

1. Start with a graph G having n nodes $a=v_0, v_1, \ldots, v_{n-1}, v_n=z$.
2. Assign load to all nodes.

   $L(a)=0$, $L(v_i) = \infty$ for $i = 1$ to n. The distinguished vertex set $S = \varnothing$.
3. **while** $z \notin S$

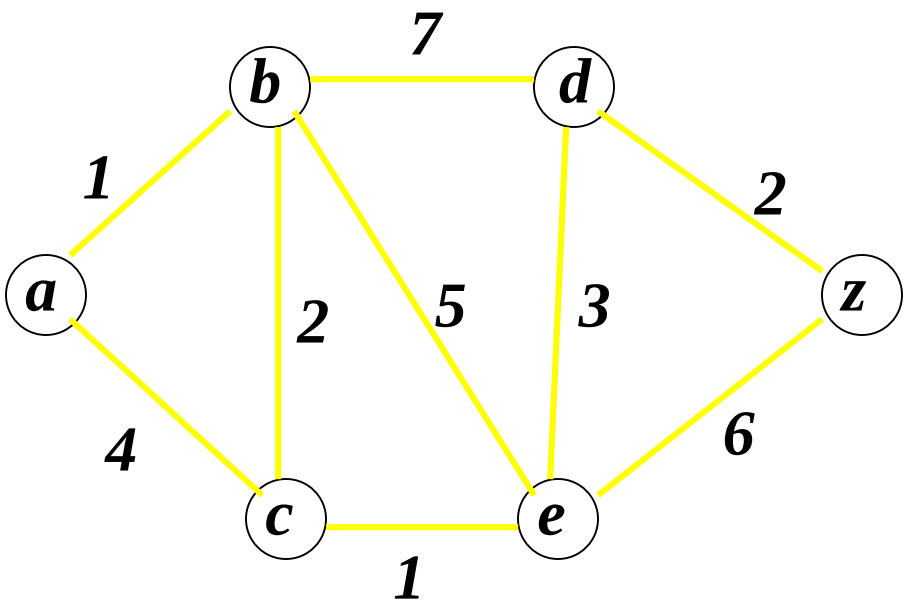
   3.1 If $u \notin S$, include node u in S if $L(u)$ is minimum out of all $L(.)$.

   3.2 **for** all vertices v not in S

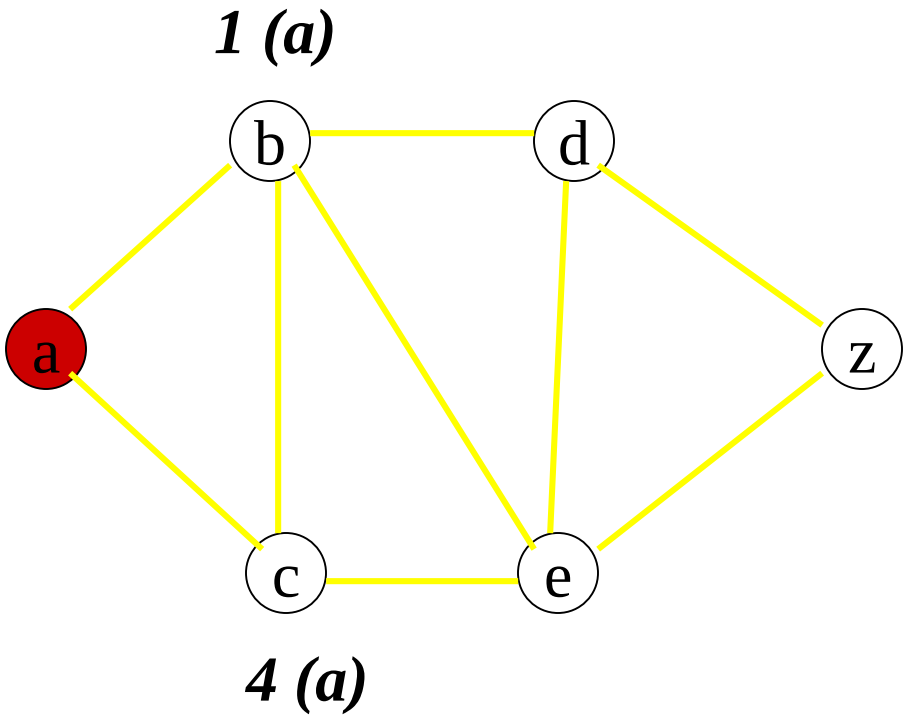
   if $L(v) > L(u) + w(u,v)$ **then** $L(v) = L(u) + w(u,v)$.

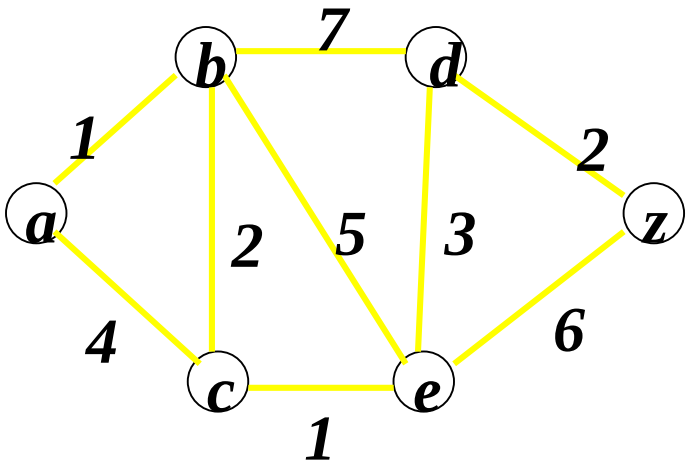
Dijkstra's Algorithm

Example:

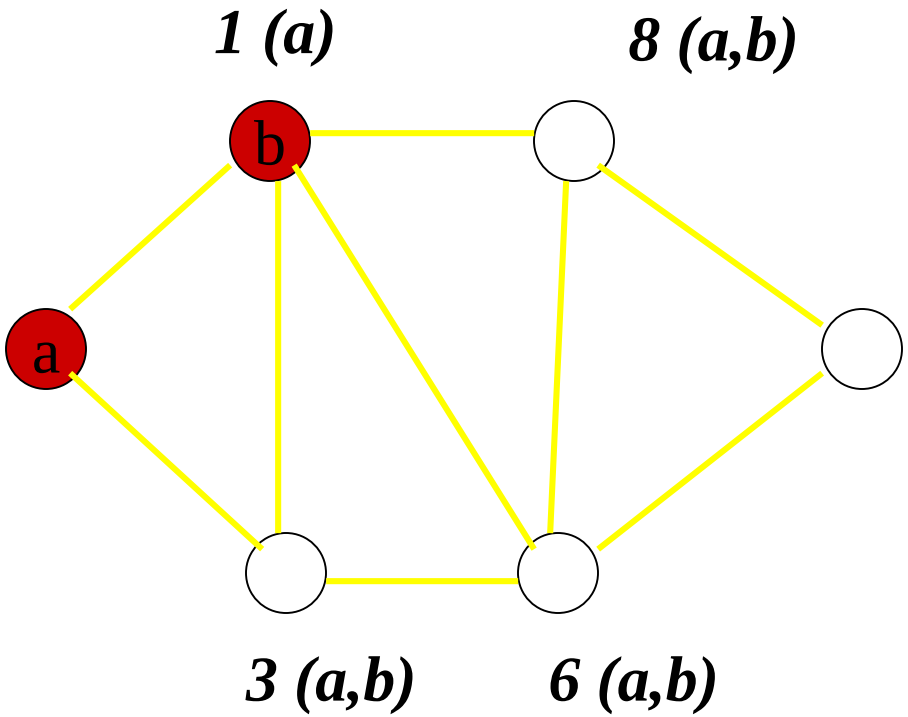


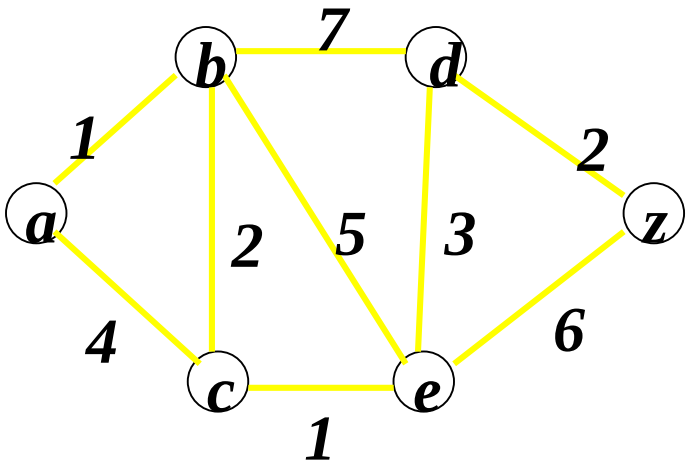
Dijkstra's Algorithm



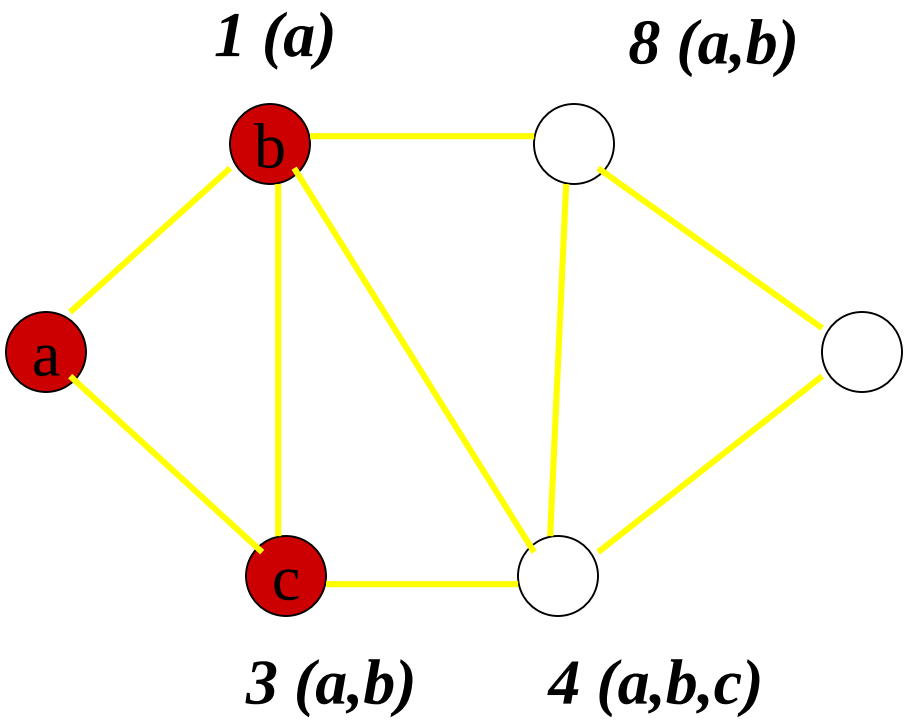


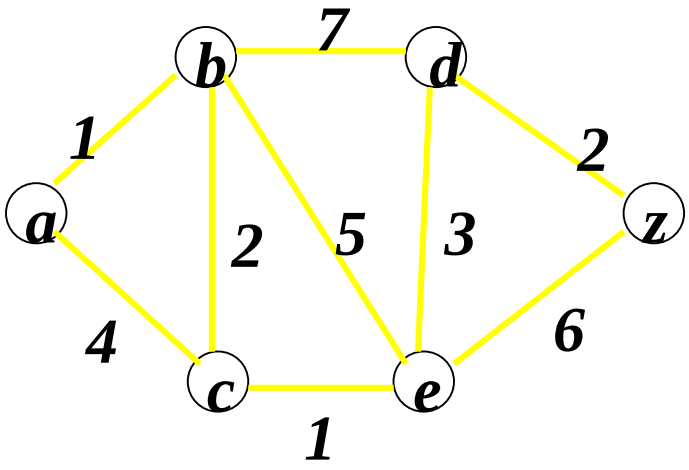
Dijkstra's Algorithm



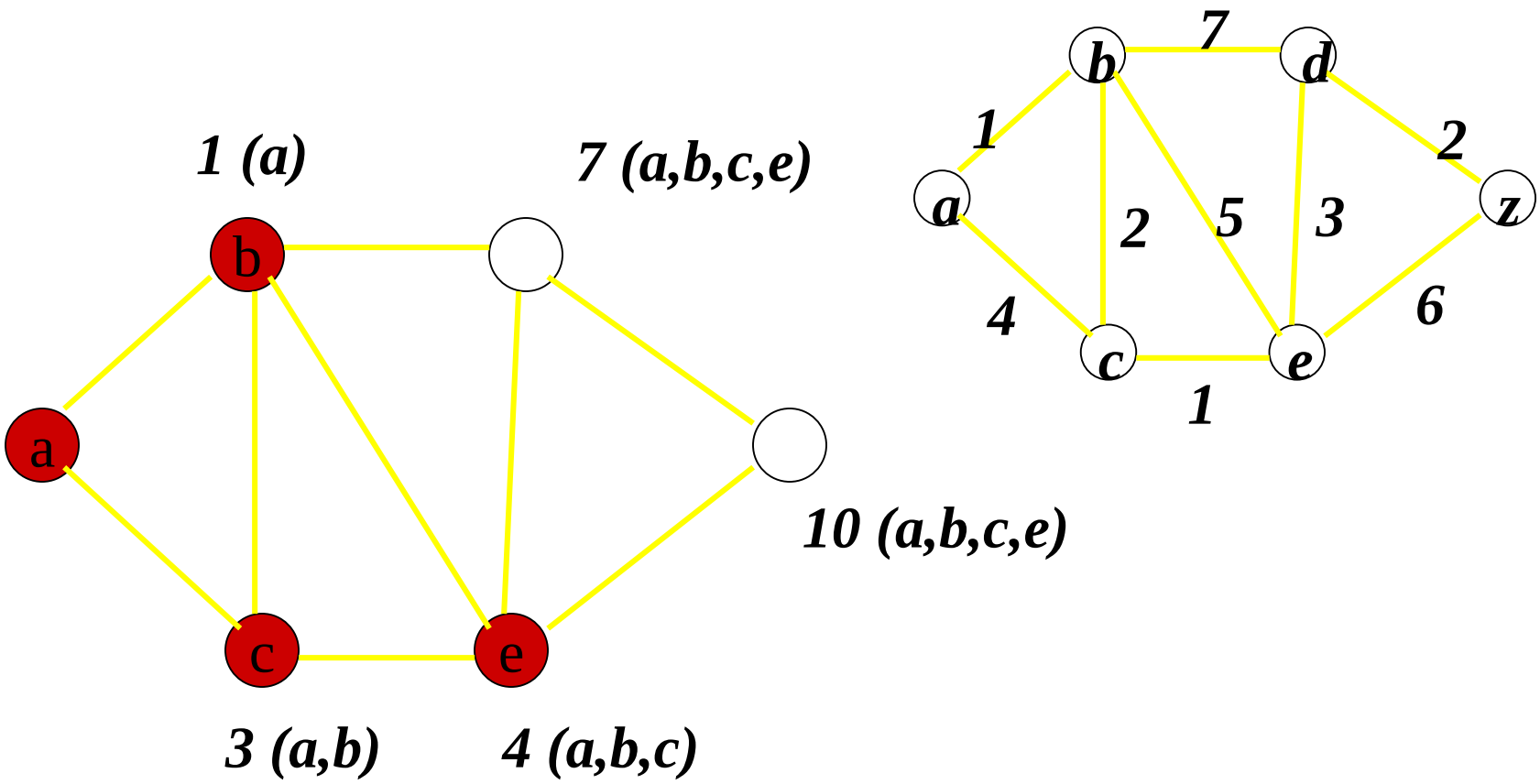


Dijkstra's Algorithm

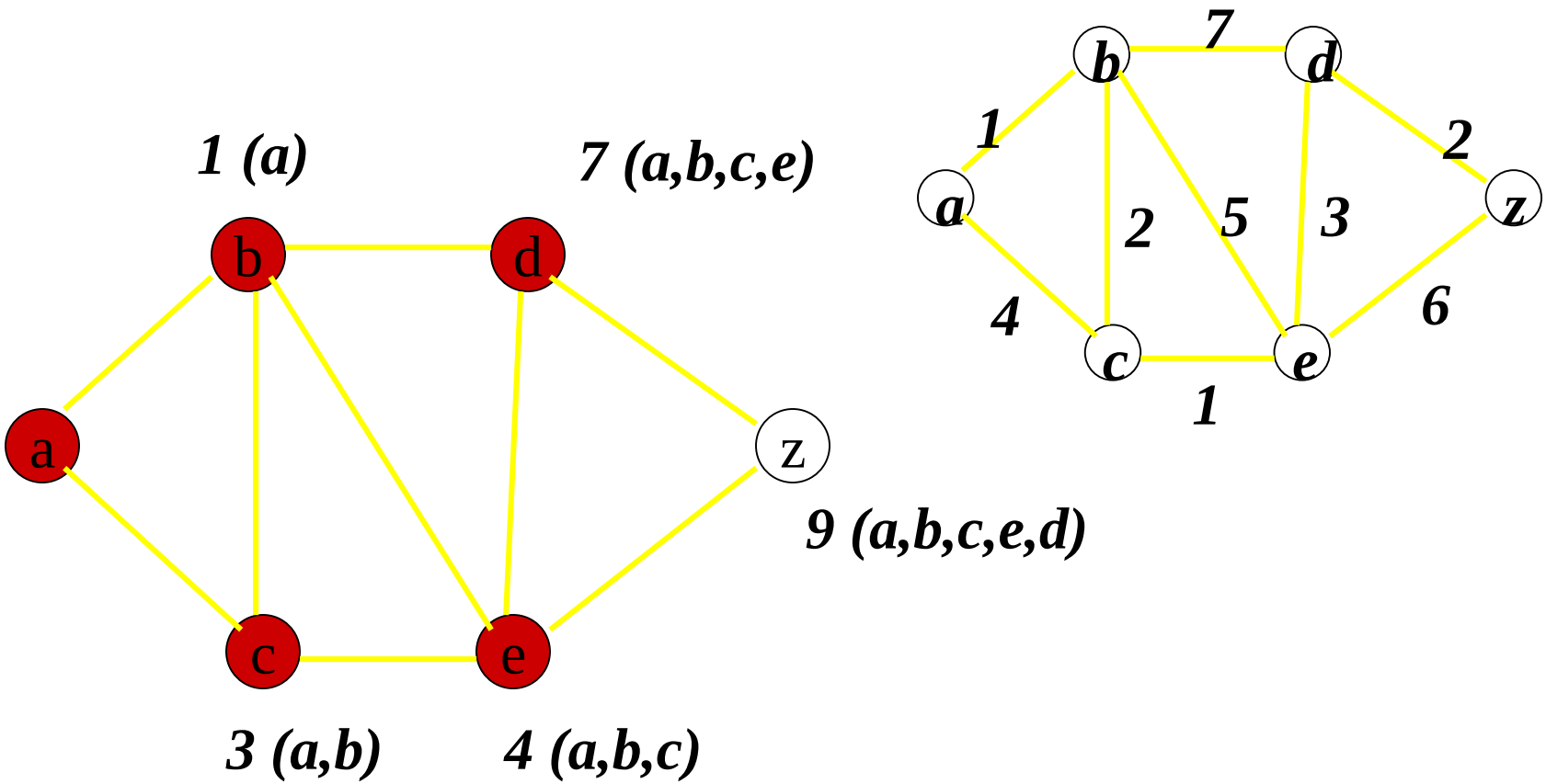




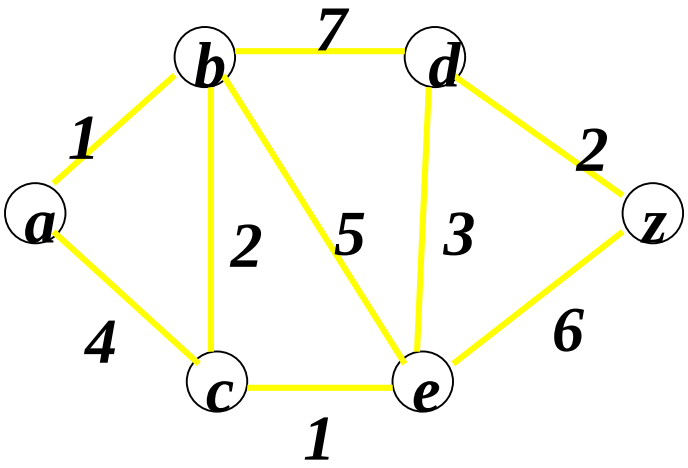
Dijkstra's Algorithm

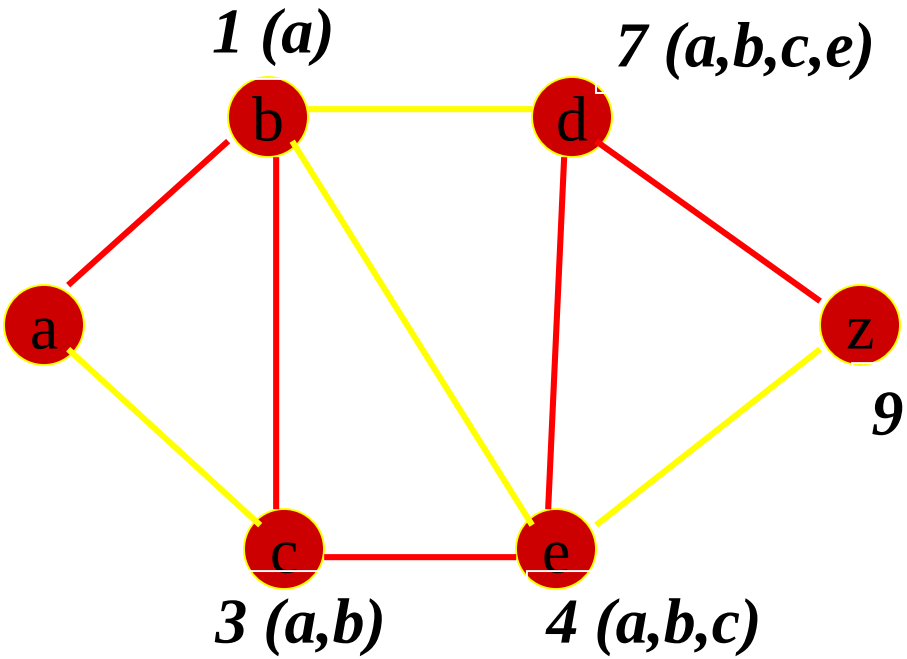


Dijkstra's Algorithm



Dijkstra's Algorithm





Dijkstra's Algorithm: Proof of Correctness

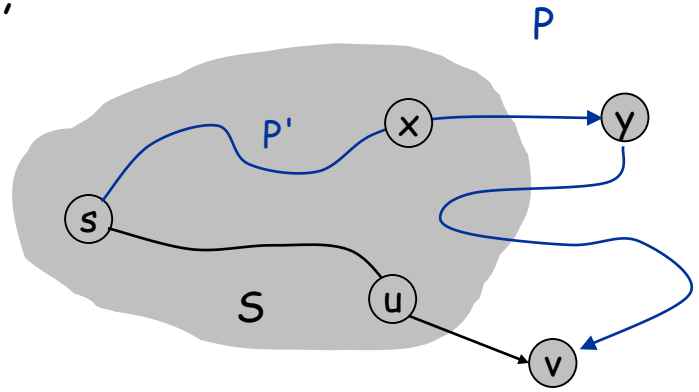
Theorem. For each node $u \in S$, $d(u)$ is the length of the shortest s-u path.

Proof. (by induction on $|S|$)

Base case: $|S| = 1$ is trivial.

Inductive hypothesis: Assume true for $|S| = k \geq 1$.

- Let v be next node added to S, and let u-v be the chosen edge.
- The shortest s-u path plus (u, v) is an s-v path of length $\pi(v)$.
- Consider any s-v path P. We'll see that it's no shorter than $\pi(v)$.
- Let x-y be the first edge in P that leaves S, and let P' be the subpath to x.
- P is already too long as soon as it leaves S.



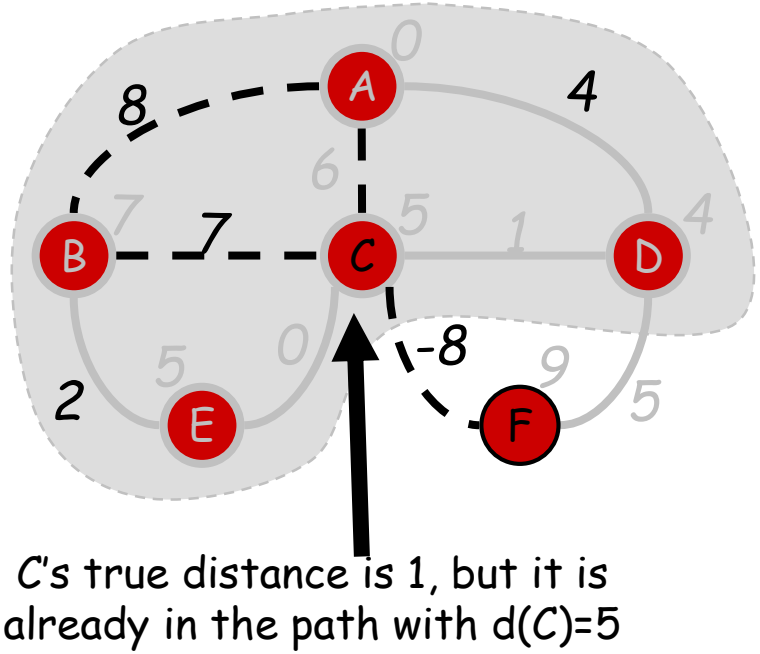
$$\ell(P) \geq \ell(P') + \ell(x,y) \geq d(x) + \ell(x,y) \geq \pi(y) \geq \pi(v)$$

- nonnegative weights
- inductive hypothesis
- defn of $\pi(y)$
- Dijkstra chose v instead of y

Why It Doesn't Work for negative-weight Edges

Dijkstra's algorithm is based on the greedy method. It adds vertices by increasing distance.

If a node with a negative incident edge were to be added **late** to the path, it could mess up distances for vertices already in the path.



C's true distance is 1, but it is already in the path with d(*C*)=5

Bellman-Ford Algorithm works for negative-wight edge

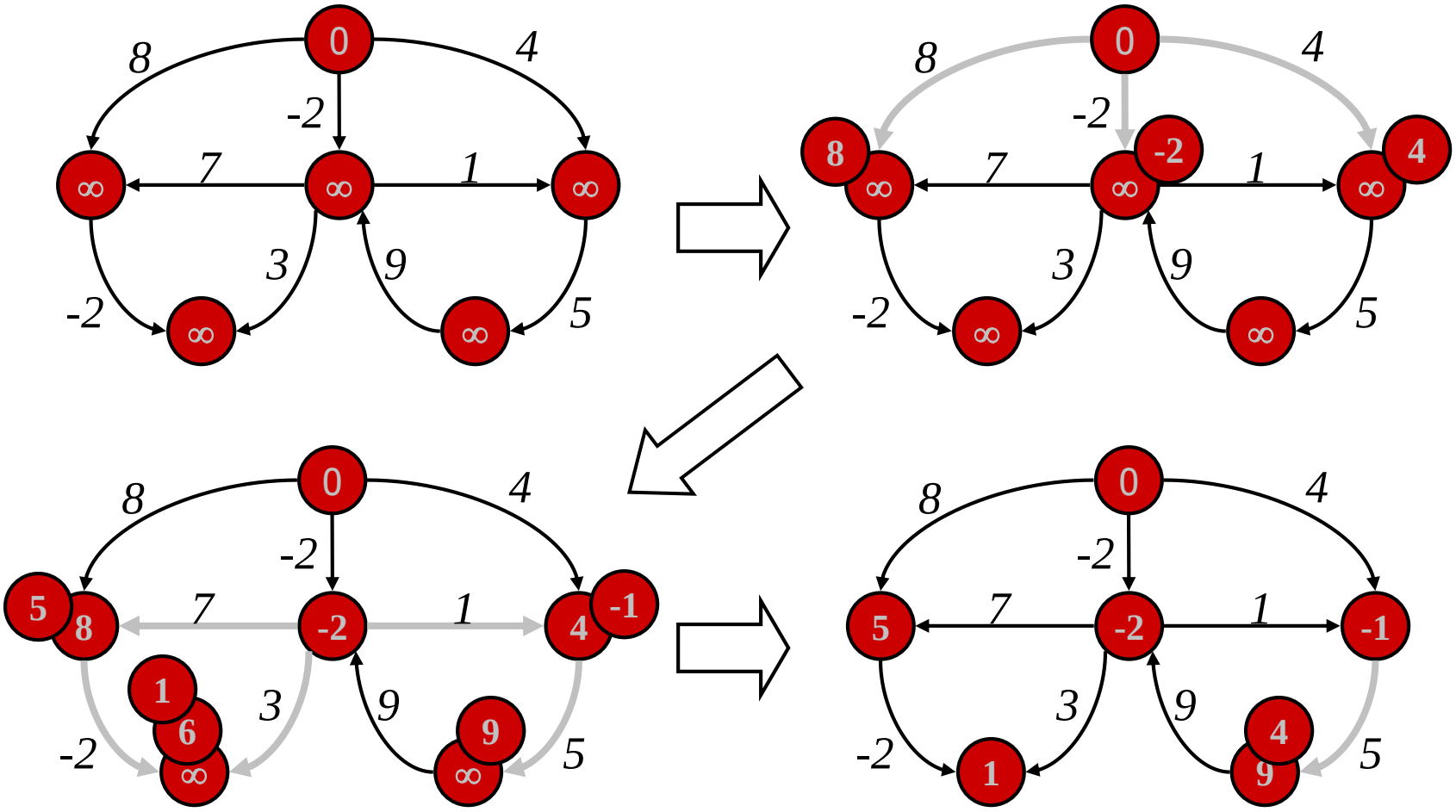
Bellman-Ford Algorithm

```
BellmanFord()
    for each v ∈ V
        cost[v] = ∞;
    cost[s] = 0;

   for i=1 to |V|-1
        for each edge (u,v) ∈ E with weight w
              if (cost[v] > cost[u] + w )
              cost[v] = cost[u] + w;
              pred(v) = u;

   for each edge (u,v) ∈ E with weight w
        if (cost[v] > cost[u] + w )
        report "negative-weight cycle exist"
```

Bellman-Ford Example



The Traveling Salesman Problem

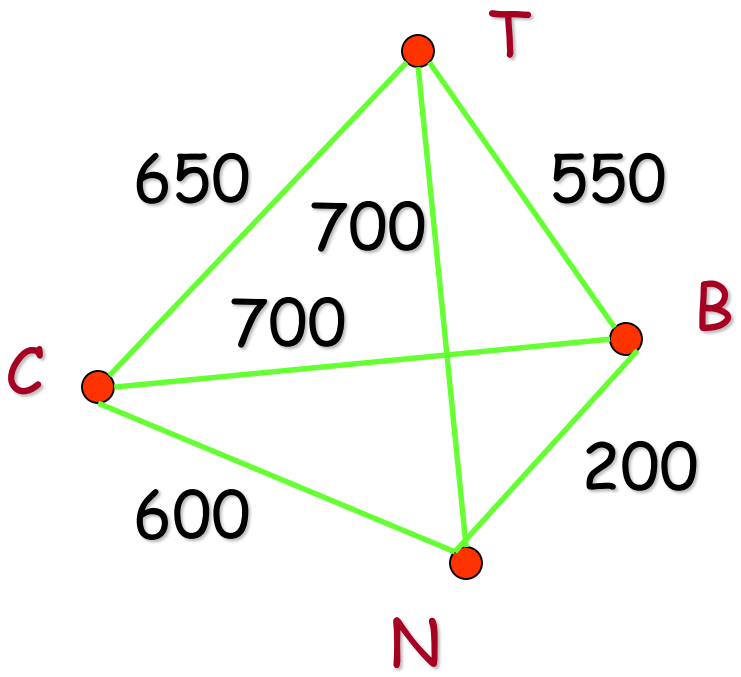
Traveling salesman problem is one of the classical problems in computer science.

A salesman wants to visit a number of cities and then returns to his starting point.

The problem then is to find the circuit of minimum total weight that visits each vertex exactly one.

The Traveling Salesman Problem

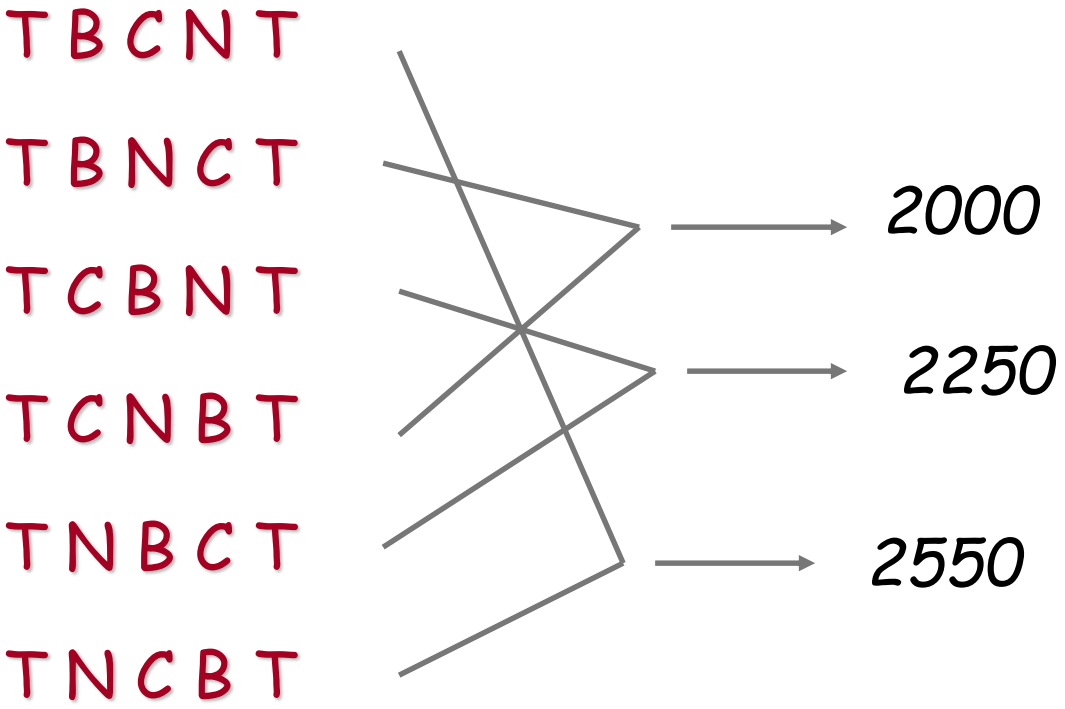
Find the shortest path that a salesman uses to visit the following cities.

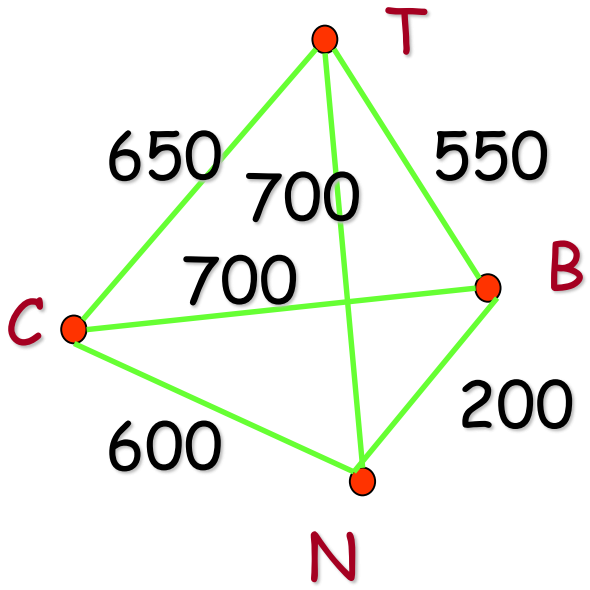


- Solution: The shortest path is T, B, N, C, T (2,000 miles).

The Traveling Salesman Problem

Consider all possible cycles and find which provides the least weight on adding all weights in the path.





The Traveling Salesman Problem

Question: Given n vertices, how many different cycles C_n can we form by connecting these vertices with edges?

Solution: We first choose a starting point. Then we have $(n - 1)$ choices for the second vertex in the cycle, $(n - 2)$ for the third one, and so on, so there are $(n - 1)!$ choices for the whole cycle.

However, this number includes identical cycles that were constructed in opposite directions. Therefore, the actual number of different cycles C_n is $(n - 1)!/2$.

The Traveling Salesman Problem

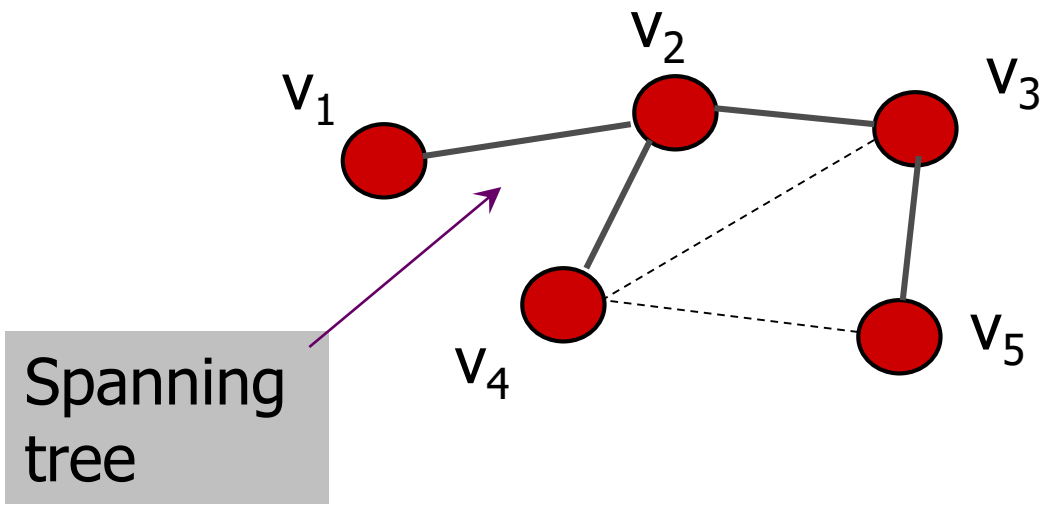
Unfortunately, no algorithm solving the traveling salesman problem with polynomial time complexity has been devised yet.

This means that for large numbers of vertices, solving the traveling salesman problem is impractical.

In these cases, we can use efficient approximation algorithms that determine a path whose length may be slightly larger than the traveling salesman's path, but produces a solution that is close to an exact solution.

Spanning Tree

Given a connected undirected graph *G*, a spanning tree of *G* is a subgraph of *G* that contains all of *G*'s nodes.



Is spanning tree unique ?

Minimum Spanning Tree

Spanning subgraph

- Subgraph of a graph ***G*** containing all the vertices of ***G***

Spanning tree

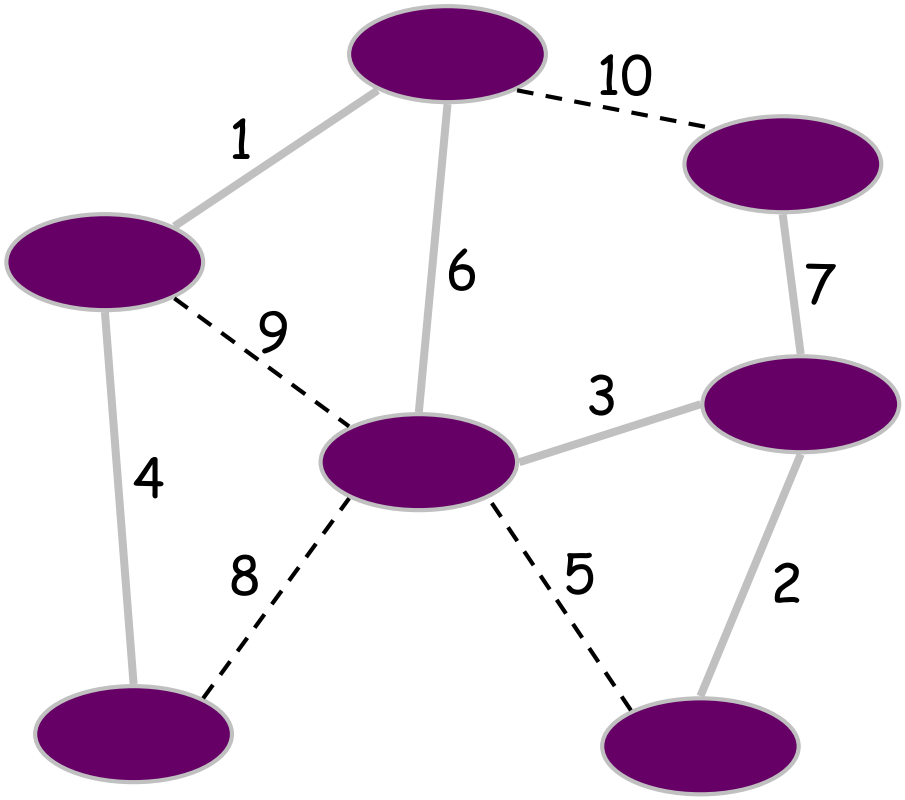
- Spanning subgraph that is itself a tree

Minimum spanning tree (MST)

- Spanning tree of a weighted graph with minimum total edge weight

Applications

- Communications networks
- Transportation networks



Minimum Spanning Tree

Consider a connected undirected graph where
- Each node u represents a country u
- Each edge (u, v) has a number which measures the cost of placing telephone line between country u and country v

Problem: connecting all countries while minimizing the total cost.

Solution: find a spanning tree with minimum total weight, that is, minimum spanning tree.

Minimum spanning tree

Given a connected undirected graph G.
Let T be a spanning tree of G.
$\text{cost}(T) = \sum_{e \in T} \text{weight}(e)$
The minimum spanning tree is a spanning tree T which minimizes cost(T)

Prim's algorithm

MST-Prim(v)

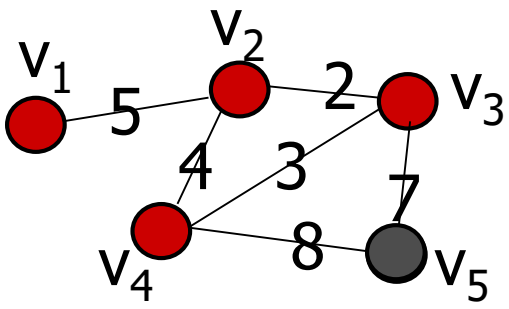
Mark node v as visited and include it in the minimum spanning tree;

while (there are unvisited nodes) {

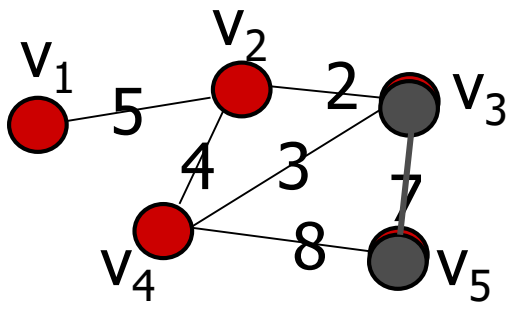
1) find the minimum edge (v, u) between a visited node v and an unvisited node u;
2) mark u as visited;
3) add both v and (v, u) to the minimum spanning tree;

}

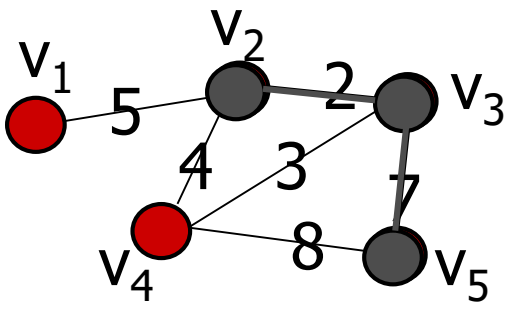
Prim's algorithm – simple illustration



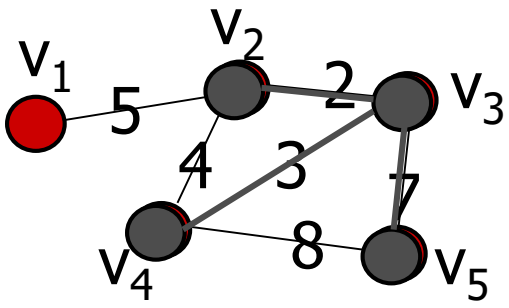
Start from v_5, find the minimum edge attach to v_5



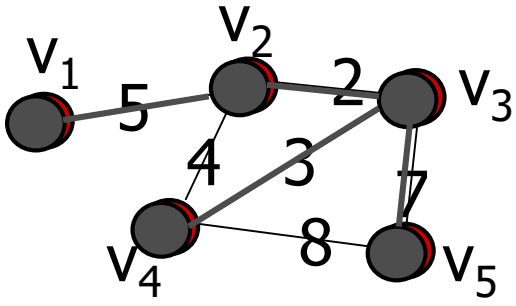
Find the minimum edge attach to v_3 and v_5



Find the minimum edge attach to v_2, v_3 and v_5



Find the minimum edge attach to v_2, v_3 , v_4 and v_5

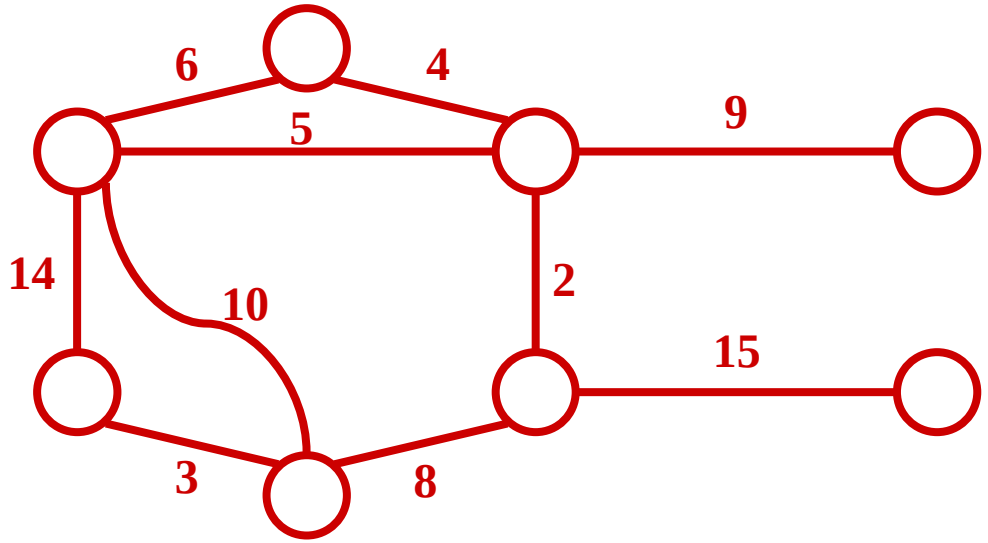


Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```

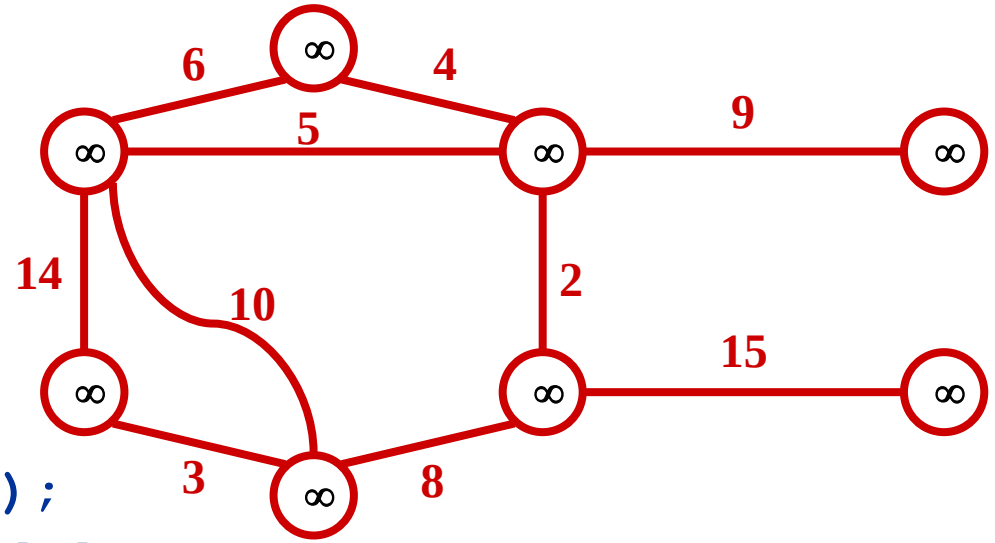
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



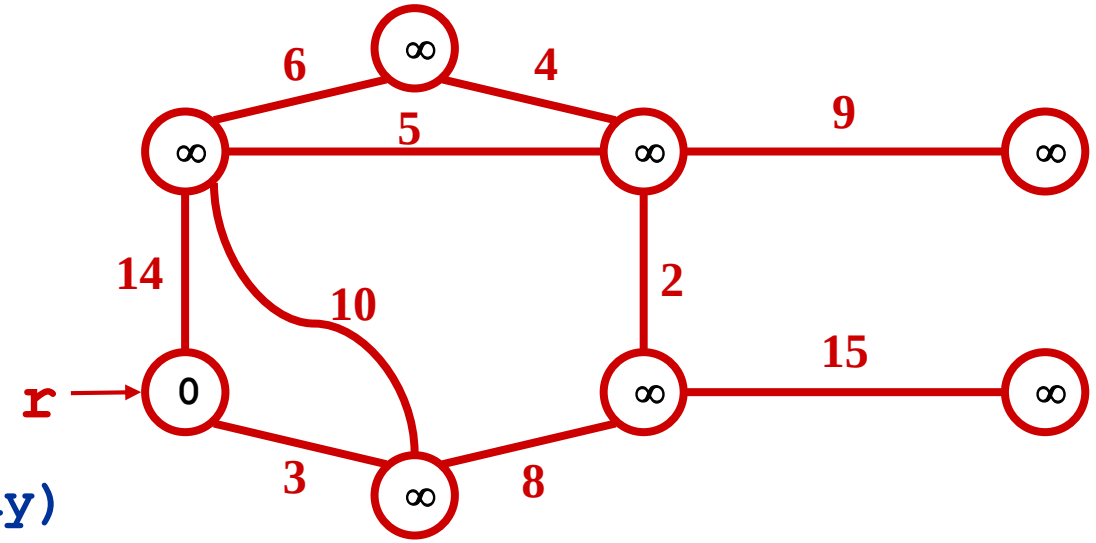
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



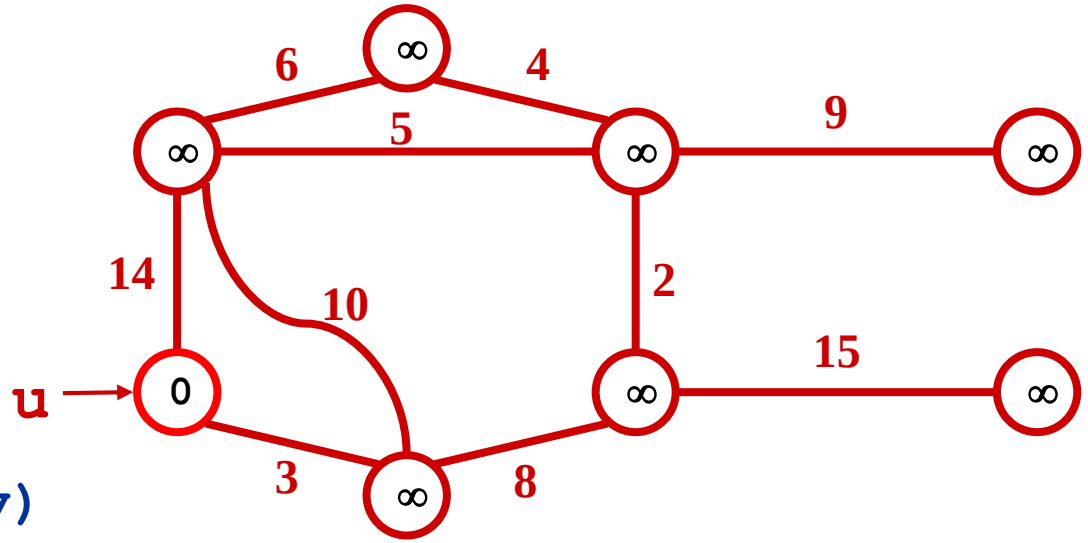
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



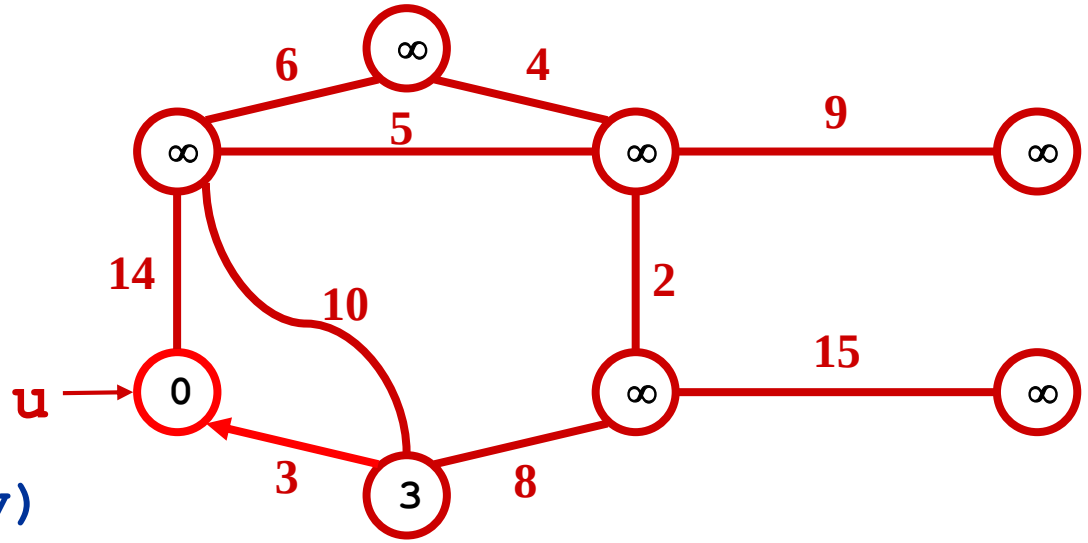
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



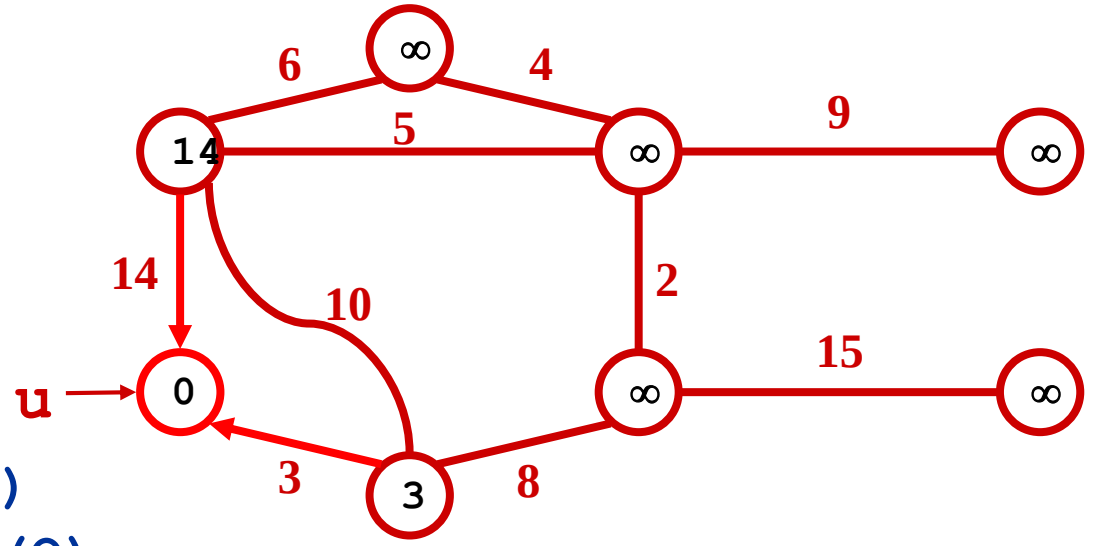
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



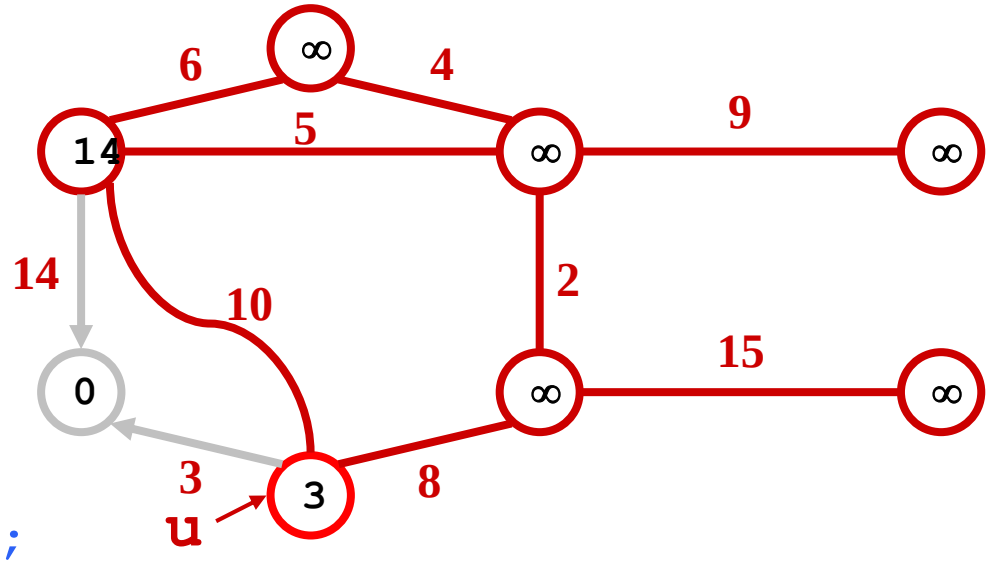
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



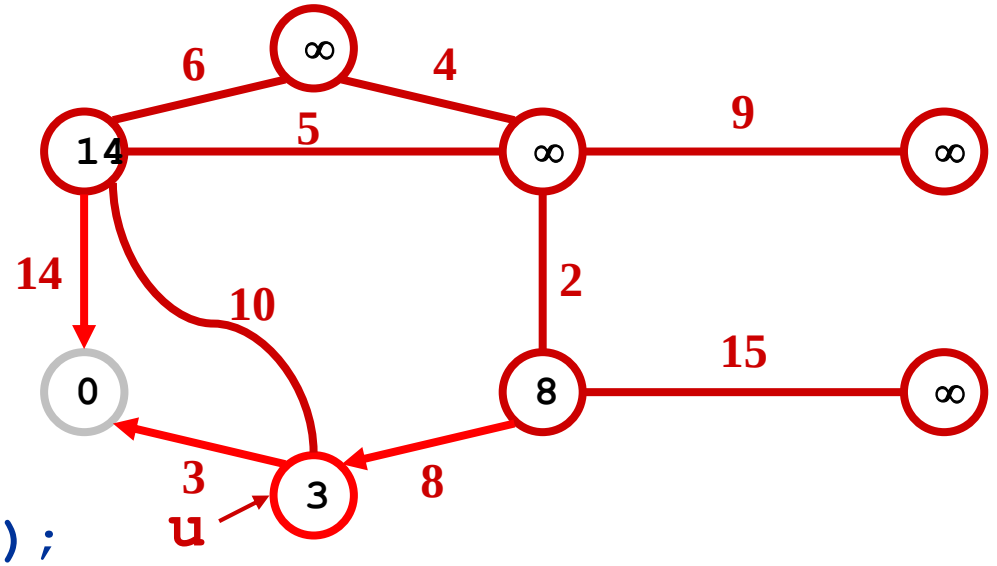
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



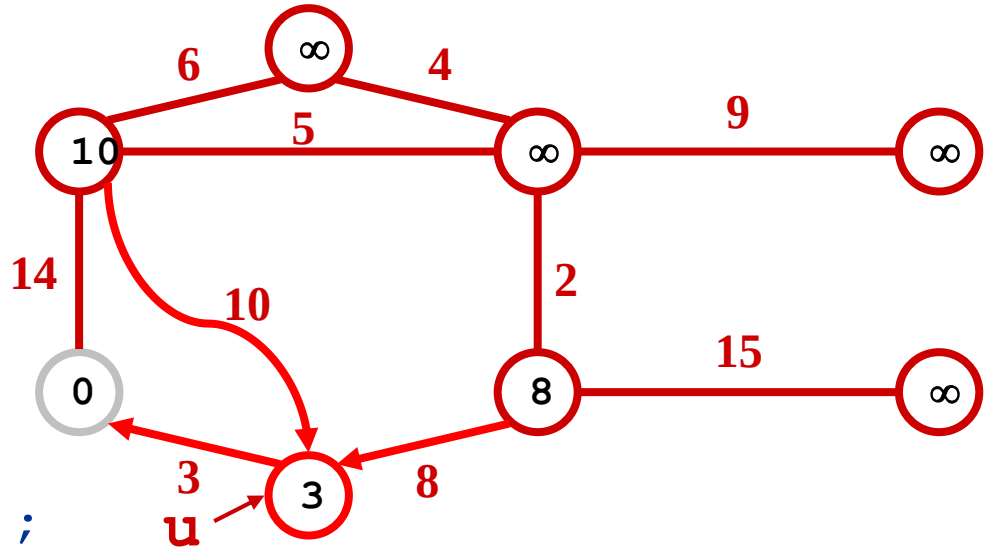
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



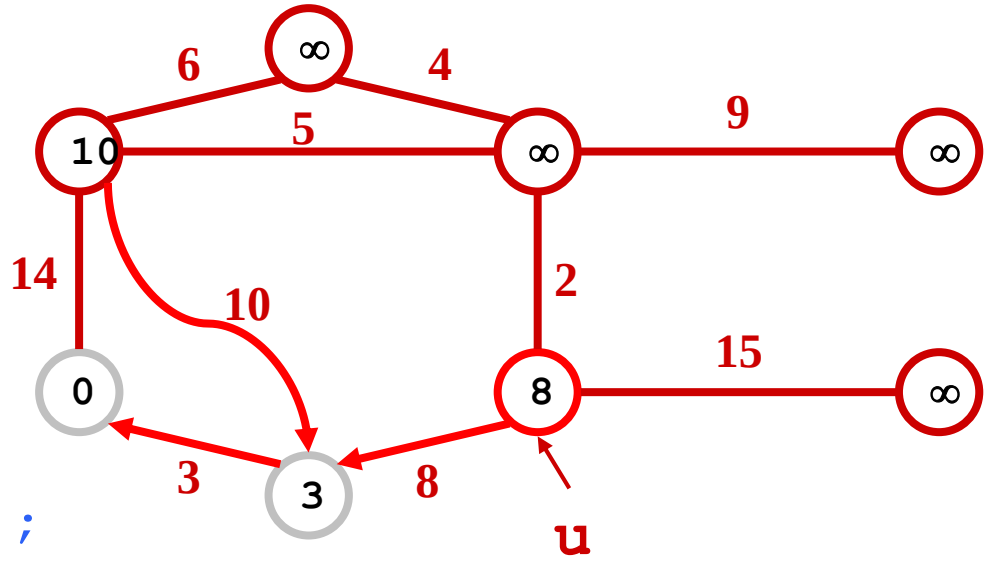
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



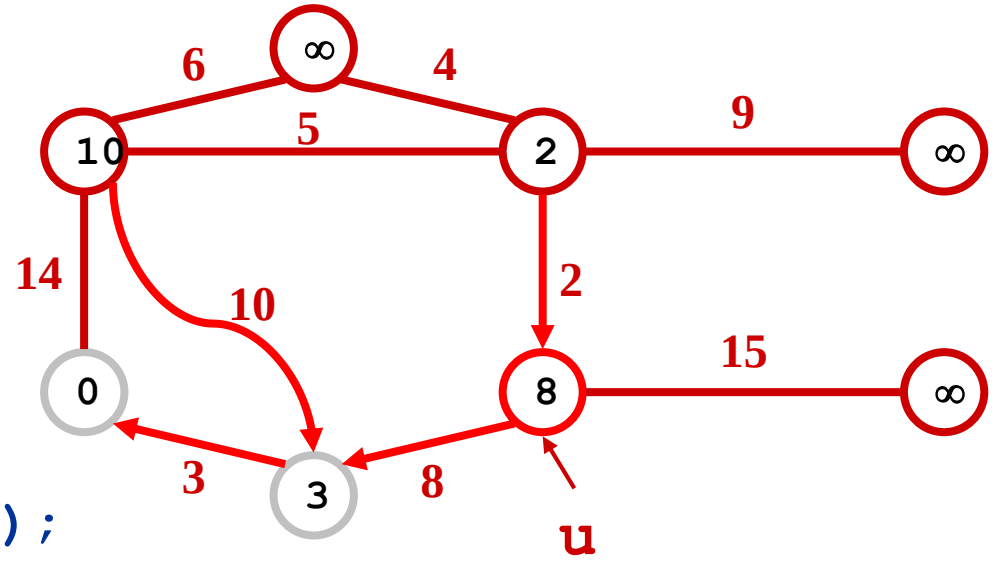
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



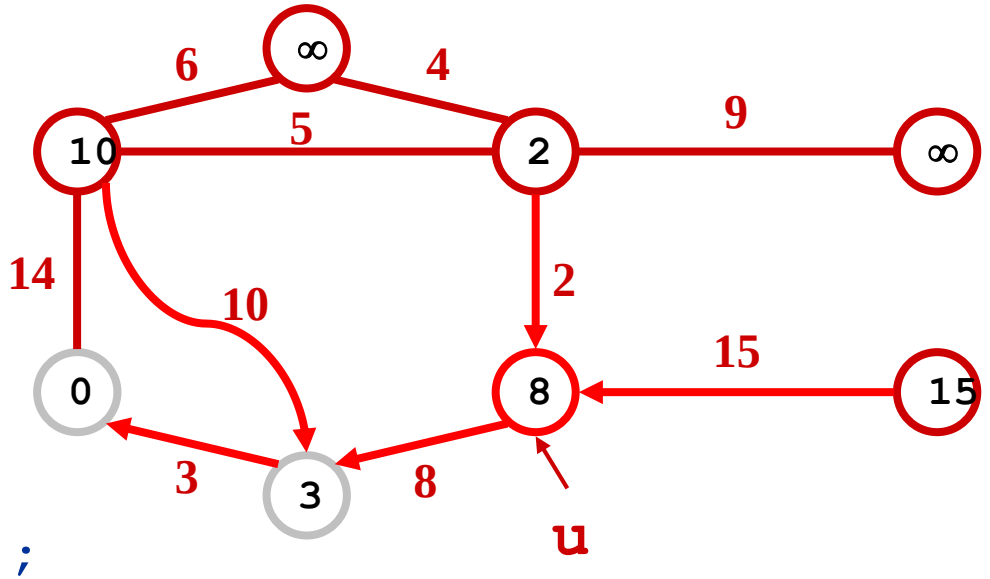
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



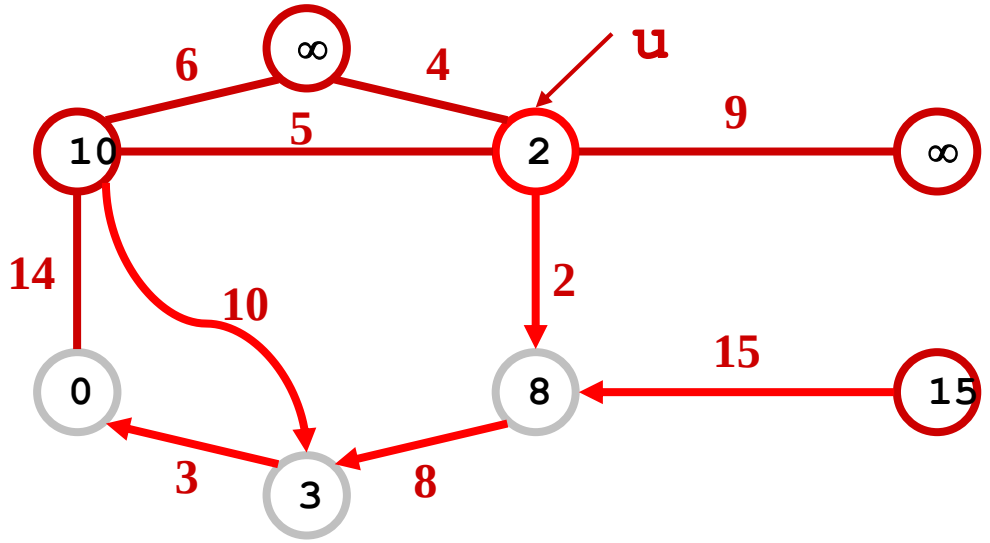
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



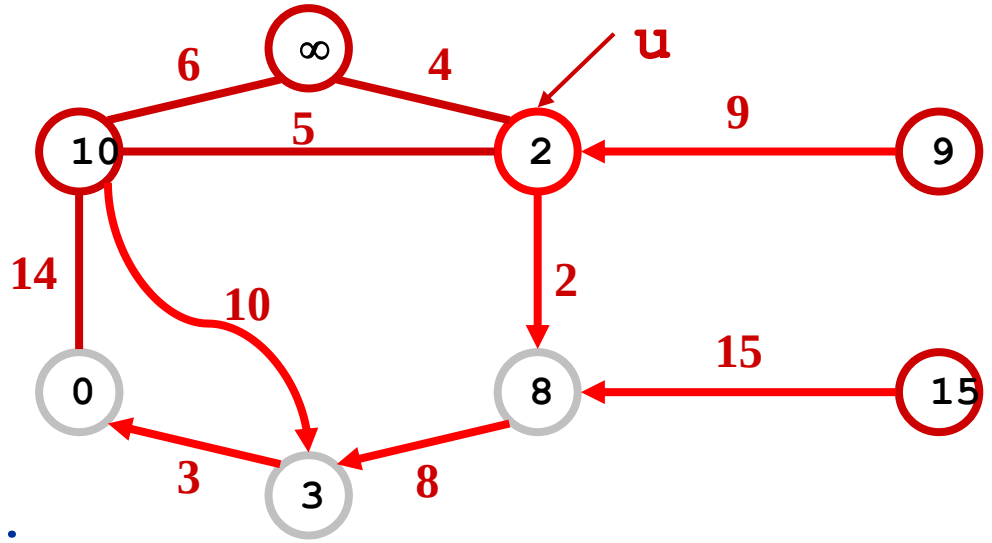
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



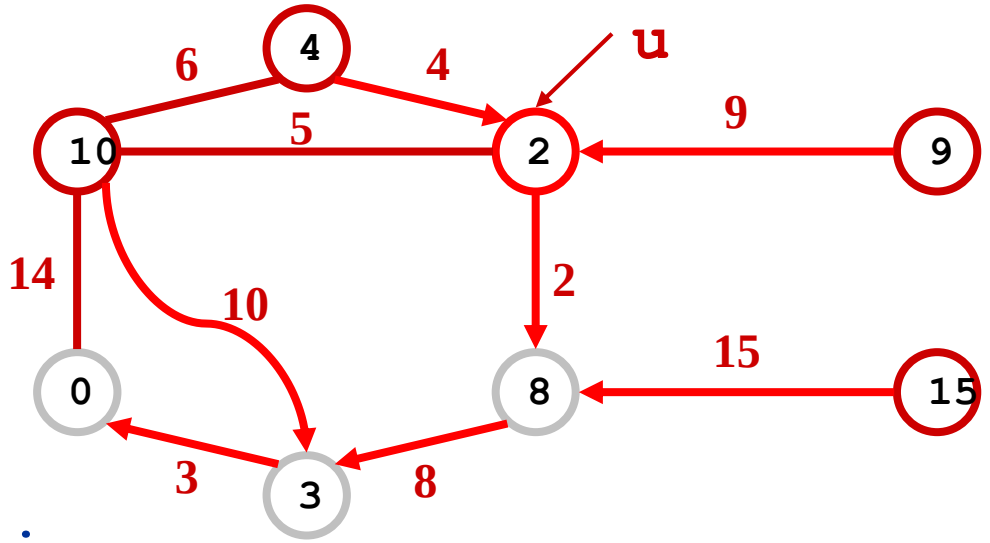
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



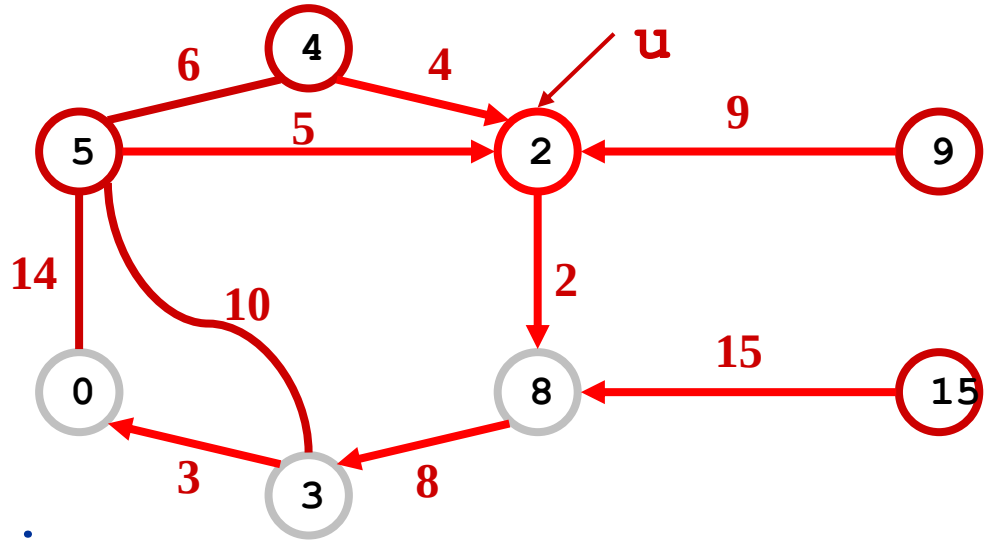
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



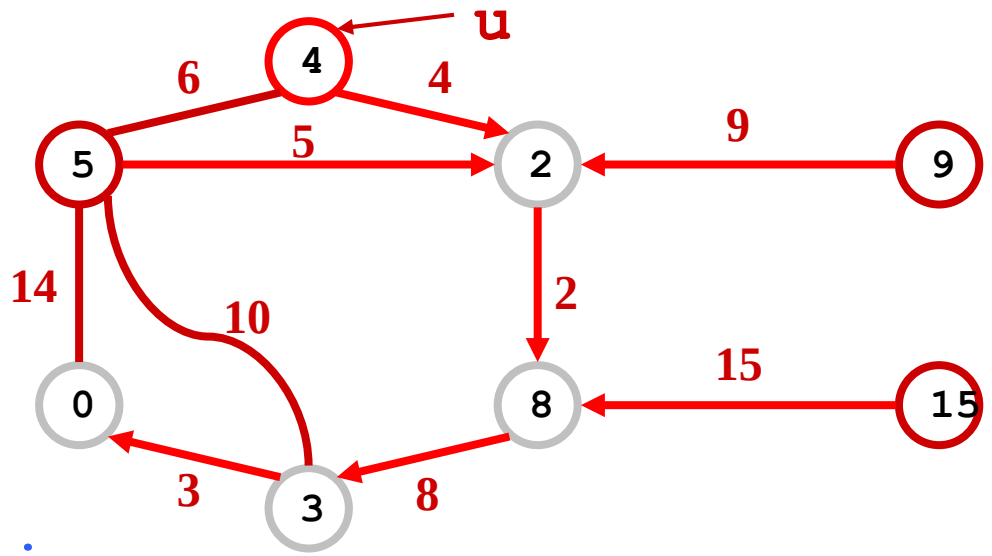
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



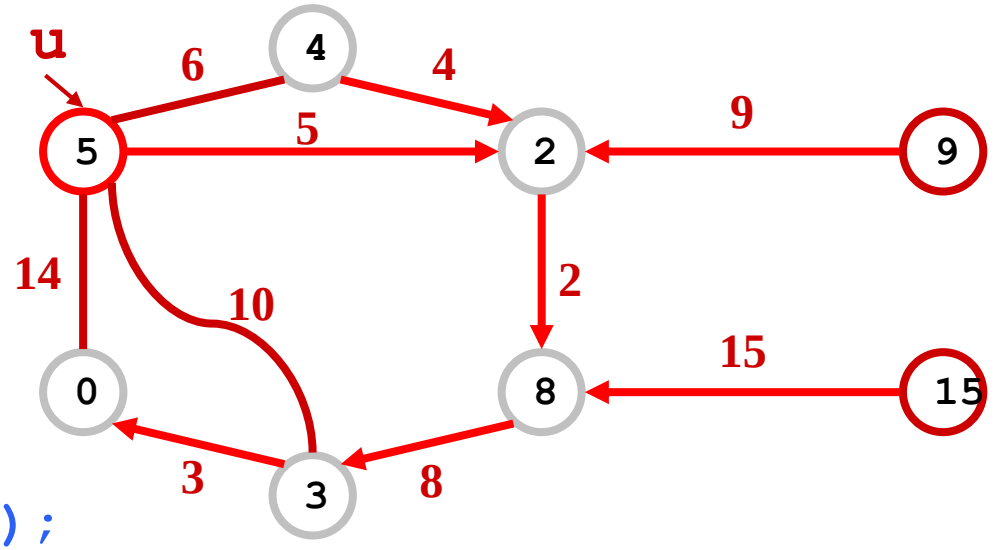
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



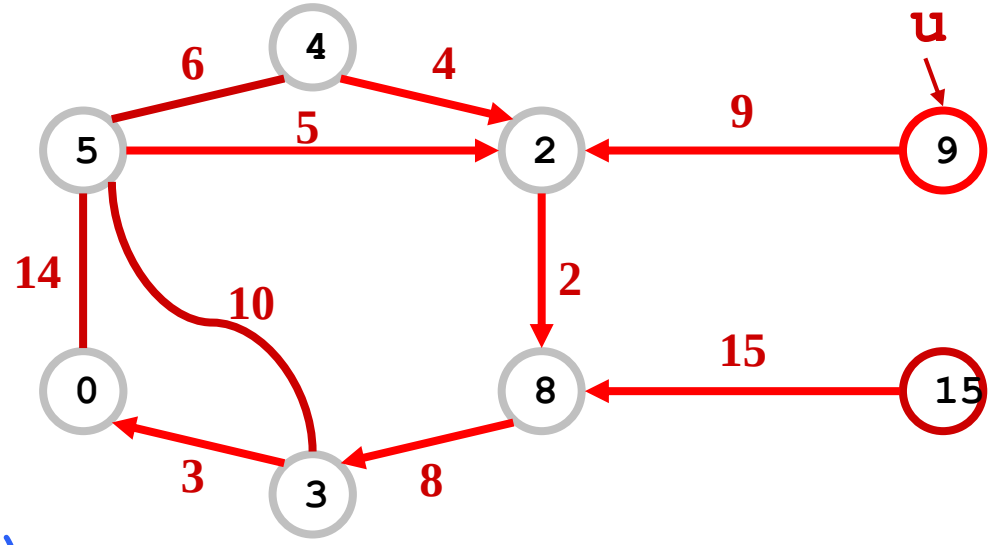
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



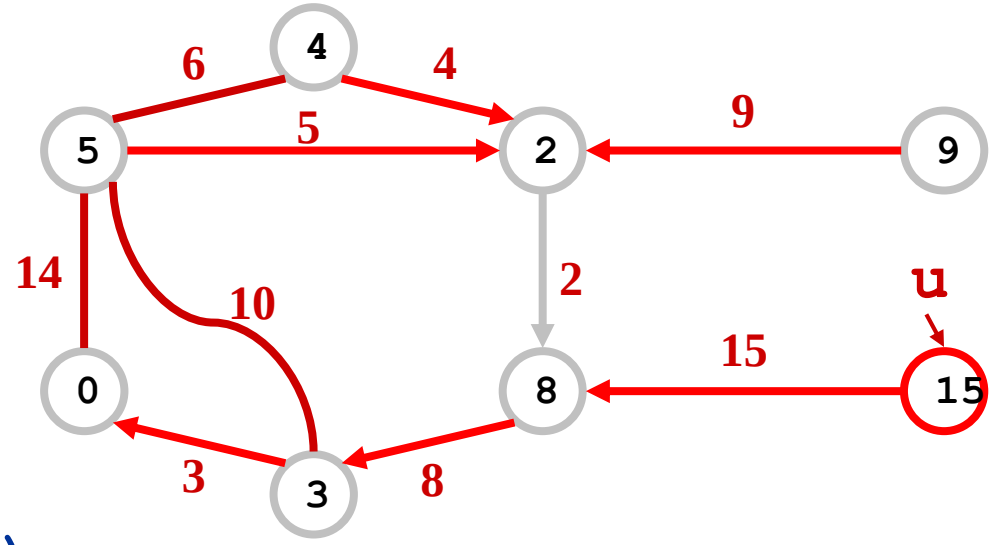
Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



Prim's Algorithm

```
MST-Prim(G, w, r)
    Q = V[G];
    for each u ∈ Q
        key[u] = ∞;
    key[r] = 0;
    p[r] = NULL;
    while (Q not empty)
        u = ExtractMin(Q);
        for each v ∈ Adj[u]
            if (v ∈ Q and w(u,v) < key[v])
                p[v] = u;
                key[v] = w(u,v);
```



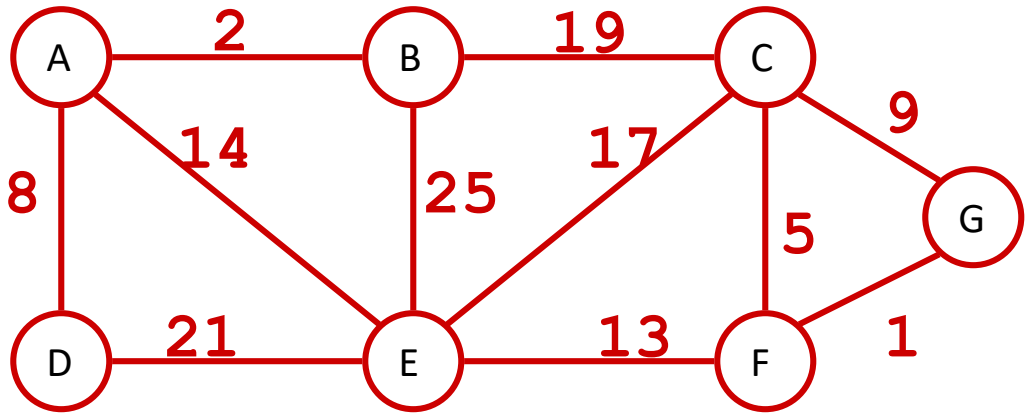
Kruskal's algorithm

MST-Kruskal

1) Given graph G, weighted connected undirected graph with n vertices.
2) T: empty graph

For i = 1 to n-1 {
    i) e = an edge of minimum weight that does not form a simple circuit in T if added to T
    ii) T = T with e added
}
Final T is a minimum spanning tree of G.

Kruskal's Algorithm - Example



Kruskal's Algorithm: Running Time

To summarize:

- Sort edges: O(E log E)
- InitCluster(): O(V)
- NewCluster(): O(E)
- Union(): O(V)

Total cost : O(E log E) or O(E log V)